

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Ecosystem Functional & Architectural Blueprint for QA Verification

Project: MentorMatch Platform

Author: Project Team Lead

Date: June 19, 2026

Version: 1.0.0

1. Executive Summary & Core Objectives

MentorMatch is an enterprise-grade peer-to-peer web ecosystem designed to bridge the gap between domain-expert mentors and aspiring students. The platform automates professional networking loops, allowing seamless search discovery, real-time interactive session orchestration, automated alert notification dispatches, and decentralized, transparent post-session community evaluations. This document acts as the definitive source of truth for all validation, test-case mapping, and boundary testing by the external QA team.

2. Distributed System Infrastructure Topology

To ensure cross-environment evaluation matching, the system utilizes a modern, loosely coupled three-tier architecture:

- **Presentation Layer (Frontend):** Angular 17 Single Page Application (SPA) driven by reactive structural layouts, strict TypeScript type-safety guards, and asynchronous client-side routing pipelines.
- **Application Logic Layer (Backend):** Spring Boot 3.2 Framework exposing standard, stateless, RESTful JSON application programming interfaces (APIs) alongside transactional data engines.
- **Data Persistence Layer (Database):** MySQL 8 / MariaDB Relational Database Management System (RDBMS) configured with strict referential constraints.
- **Security Architecture Base:** Stateful user identity tracking handled exclusively via JSON Web Tokens (JWT) passed using client-side secure storage and injected into outgoing HTTP request headers.
- **Bi-Directional Messaging Framework:** HTML5 WebSockets over the Simple Text Oriented Messaging Protocol (STOMP) standard, utilizing a client-side SockJS abstraction fallback bridge layer to ensure firewall-agnostic execution.

3. Actor Boundaries & Authorization Matrix

The testing suite must execute cross-role boundary validation across exactly three separate authenticated personas:

User Role	Authorized Functionality Boundaries
Anonymous / Guest	Access restricted to public landing paths, registration entry masks, public mentor profile discovery catalogs, and general query search parameters.
Authenticated Student	Authorized to maintain an educational profile, initiate booking requests, interact with real-time notification components, manage individual dashboard flags, and post quantitative post-session reviews.
Authenticated Mentor	Authorized to declare industry focus parameters, set hourly pricing limits, modify a dynamic skill-tag collection, process appointment requests via a life-state engine, and read aggregated performance analytical summaries.

4. Exhaustive Functional Requirements

4.1 Module 1: Session Scheduling & Lifecycle State Machine

The appointment tracking pipeline acts as a strict, sequential finite state machine. A session record must advance through individual lifecycle thresholds based on deterministic rules:

- **FR-1.1 Initialization Phase (*PENDING*):** When a Student selects an open availability slot on a Mentor's profile calendar and commits the booking, the backend must instantly generate a session record initialized explicitly to a *PENDING* state.
- **FR-1.2 Validation Requirements for Acceptance (*ACCEPTED*):** When a Mentor reviews an incoming request and selects "Accept", the outbound request payload must contain a valid, non-empty virtual meeting conference link string (e.g., a verified Zoom or Google Meet URL). If this payload string is empty or null, the backend transaction must be rejected with an **HTTP 400 Bad Request** validation code. Upon successful string verification, the status column mutates to *ACCEPTED*.
- **FR-1.3 Exception & Interruption Handling (*REJECTED* / *CANCELLED*):** A mentor must be able to actively decline an incoming pending booking, which transforms the state to *REJECTED*. An accepted active session can be terminated before execution by authorized system participants, updating the status column to *CANCELLED*.
- **FR-1.4 Lifecycle Terminal Threshold (*COMPLETED*):** Following the chronological conclusion of a 1-on-1 video call, the mentor must click "Mark as Completed" inside their panel, moving the database status flag to *COMPLETED*.

CRITICAL TESTING GATE NOTE

The transition to *COMPLETED* acts as the exclusive database flag required to unlock the review submission API for that specific session. Testers must verify no review can be submitted under any other state configuration.

4.2 Module 2: Real-Time STOMP/WebSocket Notification Pipeline

- **FR-2.1 Handshake Authentication Constraints:** The frontend application must initiate an upgraded WebSocket handshake directed to the base route `/ws`. The initial network handshake packet *must* bundle a valid, active JWT string inside the STOMP connection headers layer. Invalid, malformed, or expired token signatures must cause an immediate handshake termination on the server side.
- **FR-2.2 Targeted Asynchronous Message Routing:** When a transaction alters a session state (e.g., a Student submits a booking request → the Mentor receives an instant alert, or a Mentor flags an acceptance → the Student receives the meeting link), the application layer must parse the target recipient's unique database account primary key and route the JSON message payload directly to their dedicated channel queue destination: `/user/queue/notifications`.
- **FR-2.3 Persistent Logging and Stateful Syncing:** To preserve notification audit trails, every fired alert payload must simultaneously write a physical record to the system database tables with an unread boolean value initialized to `is_read = false`. The frontend navbar component must implement a reactive notification bell badge that subscribes directly to the `unreadCount$` data stream, updating automatically.

4.3 Module 3: Post-Session Review & Quantitative Analytics System

- **FR-3.1 Operational Prerequisite Guard Clause:** The primary API controller endpoint handling review instantiation (`POST /api/reviews/sessions/{sessionId}`) must verify the corresponding session record status prior to execution. If the database row evaluation fields return a status value anything other than `COMPLETED`, the entire transaction must be blocked with a descriptive exception message.
- **FR-3.2 Authentication Identity Guard Check:** The system must completely ignore any user ID or name parameter values passed openly inside the JSON request body payload. Instead, the backend validation service must programmatically resolve the reviewer's identity directly from the authenticated Spring Security context principal token email. If this derived account primary key does not match the exact `session.student.id` associated with that session row, the request must fail with an access-denied exception.
- **FR-3.3 Unique Relational Constraints:** The data persistence framework enforces a strict `UNIQUE(session_id, reviewer_id)` constraint layout. Testers must validate duplicate submission blocks; any consecutive attempts by an identical student user to post an evaluation for a single completed appointment ID must be blocked instantly on the database layer, returning a clean handling error response.
- **FR-3.4 Metric Aggregation & Star Layout Computations:** The mentor dashboard reviews panel must fetch its data array via a secure GET route (`/api/reviews/mentor/me`). The frontend component must execute an internal RxJS map or functional array reduction on load to calculate the mentor's collective lifetime average rating to exactly 1 decimal place, mapping the raw floating-point score safely to the template layout using structural repetition directives (`*ngFor`) to render precise interactive star configurations (★ and ☆).

5. Non-Functional Constraints & System Robustness Standards

- **NFR-5.1 Transactional Atomicity Validation:** The execution steps within `ReviewService.submitReview` (which commits a review entry and instantly triggers a secondary database notification save) must be encapsulated within a single atomic Spring `@Transactional` scope. If the database write of the notification entity fails, the saved review row must automatically roll back to prevent data mismatch states.
- **NFR-5.2 Security Boundaries & DTO Network Decoupling:** Raw JPA Hibernate mapping entities containing internal configuration variables, sensitive parameters, or password hash values must never be leaked over public network routes. All inbound data entry payloads and outbound object streams must utilize explicit Request/Response Data Transfer Objects (DTOs) from the `dto` package.
- **NFR-5.3 Cross-Database Interoperability and SQL Character Escaping:** To ensure flawless system migrations between local development engines (MySQL 8) and cloud staging platforms running strict MariaDB configurations (AlwaysData), the persistence mapping profiles must explicitly escape column attributes matching server system reserved keywords (such as `current_role` or `goals`). Column definitions inside the Java entity layers must wrap these attributes inside standardized backticks (```) to prevent runtime query syntax parsing failures across changing environments.

6. Relational Test Data Schema Baseline

Automated test automation scripts and verification inputs can reference this structural database constraint schema when creating evaluation target tables:

```
-- Architectural Reference Table for QA Test Scenario Mocking
CREATE TABLE reviews (
  id BIGINT NOT NULL AUTO_INCREMENT,
  session_id BIGINT NOT NULL,
  reviewer_id BIGINT NOT NULL,      -- Validated as Student User Foreign Key
  reviewee_id BIGINT NOT NULL,     -- Validated as Mentor User Foreign Key
  rating INT NOT NULL,
  comment TEXT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  CONSTRAINT CONSTRAINT_Score_Range CHECK (rating >= 1 AND rating <= 5),
  CONSTRAINT UNIQUE_Appointment_Review UNIQUE (session_id, reviewer_id),
  FOREIGN KEY (session_id) REFERENCES sessions(id),
  FOREIGN KEY (reviewer_id) REFERENCES users(id),
  FOREIGN KEY (reviewee_id) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```